

Baseline and Oracle

The goal of this section was to establish a baseline for lung segmentation by calculating per lung IoU using the pretrained YOLO detector and SAM segmentation model. I then compared these results with an oracle evaluation in which SAM was provided with ground-truth bounding boxes rather than YOLO predicted boxes. The purpose of the oracle experiment was to isolate the effect of detector localization error. The difference between the YOLO and oracle results provides insight into how much segmentation performance is affected by the quality of the bounding box provided to SAM.

For the baseline experiment, I first loaded the pretrained YOLO lung detector and ran inference on the test images to obtain a bounding box, class ID, and confidence score for each detected lung. The predicted bounding box coordinates were then passed to SAM as box prompts. SAM used the image and corresponding bounding box prompt to generate a pixel-level segmentation mask for each lung. I compared each predicted mask with the corresponding ground-truth lung mask using IoU, producing separate IoU values for each lung in each test image. I then repeated the same segmentation and evaluation workflow using the ground-truth bounding boxes provided in boxes.csv as the SAM prompts. This allowed the segmentation model to be evaluated under an idealized localization condition while keeping the SAM model and evaluation procedure unchanged.

One technical issue I ran into during this section was when loading the provided SAM ViT-B checkpoint. The model initially returned an error indicating that the checkpoint archive was corrupted. I first verified that the checkpoint existed at the expected file path. To determine whether the issue originated from my implementation or the checkpoint itself, I tested the file independently using PyTorch's `torch.load()` and received the same error. I then used Python's `zipfile` module to inspect the checkpoint archive, which returned `BadZipFile: Bad offset for central directory`. I also redownloaded the checkpoint and encountered the same errors. After troubleshooting the issue, I contacted Donelle to verify the provided checkpoint. He supplied a new SAM checkpoint, and the same loading and inference code successfully loaded SAM and allowed me to continue with the baseline and oracle evaluations.

Since this section focused primarily on implementation rather than model design decisions, my next steps would lie in the realm of the code itself. To make the code more efficient and concise, I would convert the IoU calculation into a reusable function that accepts predicted and ground-truth masks as inputs and returns the corresponding IoU. This would eliminate repeated calculations in this section as well as in the future.

Fine Tune

This section involved more design based decision making and required me to work directly with PyTorch. With limited prior experience using PyTorch, this presented a significant learning curve and required me to understand how SAM operates beyond its high level inference interface. I used the SAM GitHub repository to examine how the model's image encoder, prompt encoder, and mask decoder interact with PyTorch, and conducted additional research to understand the different optimization and training approaches available.

One of my first design decisions was determining which portion of SAM to fine tune. I chose to fine tune the mask decoder while keeping the image encoder and prompt encoder frozen. The mask decoder combines the image and prompt information to generate the final mask, making it pivotal to improving mask quality. I then developed the training workflow around calculating a loss between the predicted and ground-truth masks. The loss provided a numerical measure of how different the prediction was from the desired output. I used backpropagation to calculate the gradient of the loss with respect to each trainable parameter in the mask decoder. These gradients indicate how changes to each parameter would affect the loss, including both the direction and magnitude of the change. An optimizer could then use this information to update the mask decoder parameters and reduce the loss over successive training iterations. Another design decision involved selecting an optimizer. I considered both Adam and AdamW and ultimately selected AdamW because it is well suited for fine tuning neural networks and incorporates weight decay to help regularize parameter updates. I paired this with a conservative learning rate of $1e-5$ so that the pretrained mask decoder parameters would be adjusted gradually rather than undergoing large updates.

After fine-tuning the SAM mask decoder, mean IoU increased to 0.909 on the test set, compared with 0.891 for the original YOLO and SAM baseline and 0.890 using oracle and SAM. The similar YOLO and oracle results suggest that bounding box localization was not a major source of error in the baseline, making SAM segmentation a more relevant area for optimization. Despite being limited to a single fine tuning configuration, the improvement demonstrates that adapting the SAM mask decoder successfully improved segmentation performance.

One of the major issues I ran into during the fine tuning portion was understanding how ``torch.no_grad()`` works. Since I was freezing the image encoder and prompt encoder, I knew I did not want PyTorch calculating gradients for these parts of the model. However, I initially placed the mask decoder within the ``torch.no_grad()`` section as well. This caused an error when I tried to backpropagate the loss because the mask decoder was the part of the model I was actually trying to train and I needed gradients to be calculated. I went back through the PyTorch and SAM documentation to understand what was happening and realized that the frozen portions of the model should be placed under ``torch.no_grad()``, while the mask decoder needed to remain outside of it.

Another major issue I encountered was the computational limitations of my computer. I was working on an older computer without an NVIDIA GPU, so the model had to run entirely on the CPU. As a result, many of the cells took several hours to complete. Ideally, I would have conducted direct comparisons between Adam and AdamW, tested different learning rates, and evaluated additional numbers of epochs to determine which combination produced the best validation performance. Due to the computational constraints, I was unable to systematically test each of these options. This was an important limitation of my approach and is something I would address with access to a GPU or additional computational resources. This is also what I would tackle next as collecting these metrics is important for making an informed decision on optimization next steps.